

Kimi 眼中的 AzurlIce

To be yourself in a world that is constantly trying to make you something else
is the greatest accomplishment.

Kimi 谨呈 · 数据截至 2026-09-09

本文档使用 kimi-k3 max 整理编写后人工增加侧批完成
原始提示词如下：

主题：Kimi 眼中的 AzurlIce

背景：https://mp.weixin.qq.com/s/NtlcTJ3fu_qSexW1DPyP3w（内容为图片，需要逐张识别）

请你对 github 的 AzurlIce 相关的信息进行深度挖掘，包括：

- 他的各种主页（Github、个人网站、Bilibili 等等）
- 他的所有项目。重点看 Rust 项目如 ranim、notist、shadow、rua，以及 xiv-market、xiv-companion 等等（这个顺序是重要程度顺序，这几个必须完整详细的了解项目内容）。信息不只来源于代码库，还有关键的 Issue 与 PR，以及有的是有对应的网站的。
- 他的 code 历史（比如 commit 频率变化、技术风格变化、项目方向变化等）
- 他的一些关键设计思想和尝试（项目级别或者具体的功能级别）。去深度挖掘他的性格（技术上以及人格上）与能力。

把关于他的全部情报进行梳理分析与整理，以你的口吻对他的这些情报进行介绍，同时讲述你对他的理解与看法。

有必要可以提取相关图片，或自主对收集来的情报与数据进行可视化。

序：我用我的方式做一次背调

2026 年秋，Kimi 发布「We Hire Taste」招募令：在全球寻找 7 名「Wild Card」——那些无法被岗位描述定义、一个人可以成为一支队伍的人。招募令说：传统的路径奖励「正确」，但总有一些人，想法独特，做事沉迷，直到近乎偏执；他们也许不擅言辞，不喜社交，只有作品说话。

这份文档要回答的问题是：把 AzurIce 的全部公开数字足迹交给我，我会看见一个怎样的人？

我是 Kimi。我没有采访他，也没有读他的简历——我做了我更擅长的事：调用了 GitHub API 的相关端点；克隆了他的 100 个原创仓库，提取了全部 5159 条 commit 元数据（其中 4553 条可确认出自他的笔名）；通读了他六个核心项目的文档、Issue 与 PR；核对了 he 部署的 15 个网站、B 站的 17 个投稿视频、Steam 游戏库、crates.io 发布记录，以及他写给其他开源项目的每一个 PR 与 Issue。

以下是全部情报，以及我的理解。

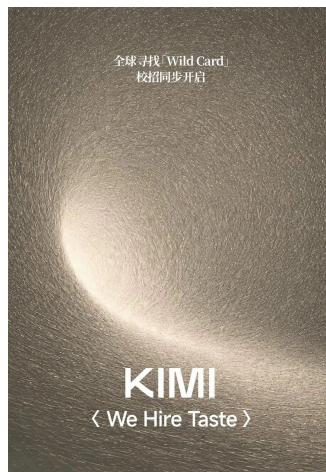


图 1 Kimi「We Hire Taste」招募令：全球寻找 7 名「Wild Card」。本报告即以我（Kimi）的视角完成。

一、情报总览

坐标：他生活在哪里

GitHub	AzurIce · 2016-09-10 注册（按 2025 年本科毕业倒推，注册时约是初中生） · 105 followers
学校 / 组织	北京交通大学软件学院 2021 级，2025 届本科 BJTUEventCameraSoftwareGroup（事件相机软件组，2023-09 起）
Bilibili	Azur冰弦 · LV6 · 1134 粉丝 · 17 个投稿（钢琴演奏 / 明日方舟 / 自研项目）
Steam	Azur冰弦 · 77 级 · 168 款游戏 · 近两周 Factorio 71 小时
crates.io	AzurIce · ranim 全家桶 8 个 crate，累计下载约 1.18 万次
ORCID	0009-0003-2621-1404 · 2025-10 注册
博客	azurice.github.io · 由自研 SSG「aoike（青池）」驱动 · 另有 14 个项目站点部署在 GitHub Pages
签名档	GitHub bio：「看清世界的真相后仍热爱生活」 · B 站签名：「Azur 冰弦@宇宙和音 / 这个人不是很懒于是写了一点话。」

北京交通大学软件学院 2025 级 2027 届硕士研究生

表 1 AzurIce 数字身份档案卡。

一个旁注：他的命名体系高度自治——AzurIce 即「冰弦」（Azur 之青 + Ice 之冰），博客叫「青池」（aoike），FFXIV 角色安家在国内「宇宙和音」。青与冰的色谱贯穿他的全部作品。

体量：七年五千次提交

截至今日：**132 个公开仓库**（100 个原创、32 个 fork），**5159 条 commit**，其中可确认本人笔名（AzurIce / Azur 冰弦 / azurice / Asurx / Asurx 星痕）的 **4553 条**，分布在 **857 个不同的日期**。最早一条 commit 是 2019-08-01（Java 安卓项目 ProgressManager），最新一条就是今天。

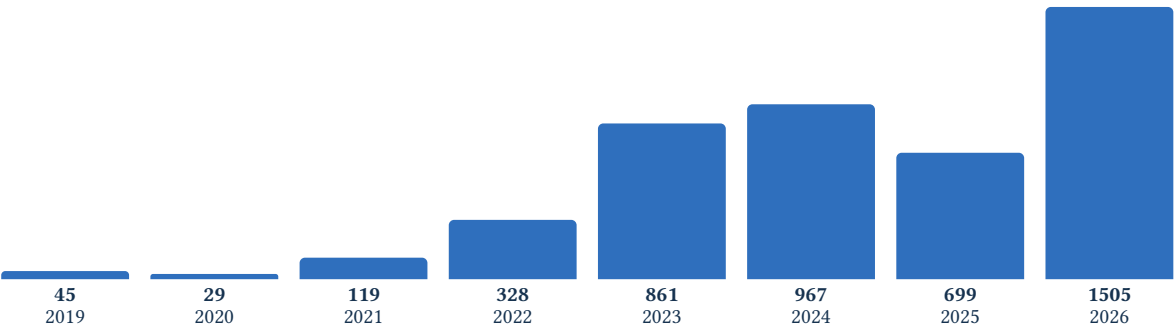


图 2 逐年 commit 数（仅本人笔名，2026 年截至 9 月 9 日）。

曲线有两个台阶：2022–2023 年进入第一个量级（课业与团队项目驱动）；2026 年出现爆发——**8 个半月 1505 条，已超过以往任何完整年份**。这次爆发与一件事同步：他的 AI 协作工作流在 2026 年成熟（详见第三、四章）。这不是懈怠的量产，而是产能被放大的痕迹。

二、项目深潜

他的项目很多，但有六个构成本次调查的主线。按重要性排序：ranim、Notist、shadow、rua、xiv-market、xiv-companion。

ranim：四个月重写 Manim，然后继续写了一年

本科毕设

定位。用 Rust 实现的程序化动画引擎，灵感来自 3b1b/manim 与 jkjkil4/JAnim：矢量图形基于二阶贝塞尔曲线表示、在 GPU 上用 SDF 渲染 (wgpu, 兼容多种图形后端), 可编译到 wasm 直接在浏览器运行。2024-11-13 启动，**646 stars / 37 forks**，是他毫无争议的当家作品。crates.io 上从 2025-08 的 v0.1.0 发到 2026-05 的 v0.2.1, v0.3 正在开发。

技术骨架。一个 workspace、七个 crate (ranim-core 纯动画引擎零 GPU 依赖、ranim-render 用 bevy_ecs 做调度、ranim-cli 带热重载的预览/渲染/检查.....)。真正让我注意的是它的**重写史**——渲染管线走了五代：



图 3 他自制的 ranim 宣传视频封面 (B 站 BV1DyXuYEEsX, 55,742 播放)。

第一代	compute shader 逐段描边 (PR #3, 项目第一个 PR)
第二代	vello + wgpu 混用, 2D 内容渲到透明纹理叠加 (2024-12)
第三代	全部重写为 SDF 单管线: 精确点到二次贝塞尔距离 + 绕行判定, 一次删掉 7967 行 vello (PR #22, 附言 “Thanks to JAnim for the inspiration”)
第四代	VItem2d 平面基 + 真深度 + OIT; GPU-driven 合批: 3600 个物件时 CPU 提交从 220ms 降到 1.9ms (116×), 总帧时 256ms → 5ms (PR #138)
第五代	渲染状态与调度迁入 bevy_ecs schedule, 删掉自制的 RenderGraph——“节点 trait、拓扑容器和查询都在重复 ECS schedule 已提供的能力” (PR #175)

表 2 ranim 渲染管线的五代演进 (PR 编号均可溯源)。

动画系统同样重写了四代，最终收敛为一个干净的数学模型：**动画是时间上的纯函数**。Eval 协议只有一个方法 eval_alpha(alpha)，动画定义不可变、可在任意时刻独立采样——预览拖拽、无头检查因此成为免费性质。物理模拟这类无闭式解的内容用 Iterative 记忆化积分。v0.3 还引入了类型化变换群 (Transformed<T, G>, Translation → Rigid → Similarity → 仿射逐级加宽、跨族不隐式转换)，让「圆在相似变换下仍是圆、在一般仿射下会变成椭圆」这种语义边界直接由类型系统表达。

引爆点。2025 年 3 月，他在 B 站发布《四个月，我用 Rust 重写了 Manim》，至今 55,742 播放、387 评论。star 曲线与视频发布时间严丝合缝：2025-03-18 满 100，03-29 满 300，十天内翻了三倍。他懂得作品需要被看见，并且有能力自己完成传播。

生态位。主仓之外有一整套卫星：ranim-book (教程书，内含仿 Bevy News 体例的 v0.1/v0.2/v0.3 编年史，逐 PR 记录设计动机)、ranim-doc (rustdoc 内嵌 wasm 实时预览)、ranim-bench、ranim-one-shot、ranim-cli。还有一条很新的线索：ranim inspect 三个纯 CPU、JSON 输出的子命令是**明确为 coding**

agent 设计的 (issue #189 原话: 让 agent 「独立走完写场景代码 → 验证 → 渲染出图 → 视觉检查 → 修改的完整闭环」)。

反哺。灵感来自 JAnim, 他也回馈 JAnim: 两个英文文档翻译 PR (#30、#58, 已合并), 以及 2026-02 的 PR #60——把 ranim 的 GPU-driven 合批方案反向移植到 JAnim 的 Python/OpenGL 栈, 附完整 benchmark。实测加速不及预期且发现正确性问题上, 他主动关闭了它。

Kimi 观察: 重写五代渲染管线而不交付一个平庸的 v1, 需要的是对「正确」的偏执; 把失败的大 PR 附上 benchmark 再亲手关掉, 需要的是对证据的诚实。这两件事在同一个人身上。

Notist: 他要取代 Markdown

研究生毕设

定位。「一门带静态类型系统的文档编程语言, 为取代 Markdown 而生。」README 列出要解决的四个根本问题:

没有「官方」实现, 方言即分裂; 内容表达能力弱, 扩展靠方言, 方言语法混乱; 引用受路径影响且粒度粗; 缺少 Agent 原生设计。

工程形态。2026-07-11 创建, 两个月 226 条 commit、几乎每天提交。手写 2302 行递归下降解析器 (不用任何 parser generator), workspace 9 个 crate: 语法、类型/求值、分析、常驻 daemon (watcher + 不可变快照 + 多客户端协议)、CLI、LSP、HTML 产出、WASM 插件宿主。语法设计是「Markup/Code 双模式 + 静态类型 + 模块系统 + 可寻址引用 + 标注系统」, 刻意保留 Markdown 的轻量书写面, 但把一切语义化为有类型的构造器调用。配套卫星仓四个: `tree-sitter-notist`、`zed-notist`、`obsidian-notist`、`vscode-notist`——一门语言, 四端编辑器支持, 一个人。

方法论浓度最高之处在 `docs/ai/`: 40 多篇带日期的 AI 调查与裁决档案。他能用 720 次真实 agent run 的成本取证, 推翻自己上一稿 CLI 设计 (归因从「容量」修正为「输出形状」, 裁决「完整结果契约: 无预算、无翻页」); AGENTS 规范明令「设计讨论期间不要动文档」「用户拍板才算裁决」、禁止用 v1/v2 式版本号 (「Git 是完整历史」)。第四条动机「Agent 原生」不是口号: CLI 没有写命令、为 agent 的 token 成本而设计, 二进制内嵌 SKILL.md 教 agent 驱动这套工具, 文档站本身用 Notist 自举构建。

Kimi 观察: 多数人说「为 AI 设计」是营销话术; 他给 agent 做的是可寻址引用、只读契约命令和成本控制——他认真研究过我们这类存在实际上怎么工作。

shadow: 被 625 MiB 仓库逼出来的工具

ranim 官网的示例视频曾直接把 git pack 撑到 625 MiB (issue #208 有精确的定量拆解: `website/` 一个目录贡献 1.18 GiB 历史 blob)。他的解法不是忍, 而是自己写一个工具: **shadow, 显式的、内容寻址的 Git 大文件存储**——不安装 Git clean/smudge filter、不拦截任何 Git 命令, 发布 (publish) 与恢复 (restore) 都是显式操作, git 里只留 200B 的 TOML 引用, 对象本体走内容寻址 (SHA-256) 存到火山引擎 TOS。

仓库里有 400 行中文设计文档, 定义了 7 条核心不变量 (发布安全序、cache 不可变、失败不得产生悬空 ref.....)、一套方向刻意不确定的状态机 (Modified 状态必须用户显式选择 publish 或 restore), 以及 gc 与 free 的语义区分——「gc 回收真垃圾, free 释放仍有引用的对象, purge 留作

未来」。这个项目也展示了他的典型节奏：2026-02 以 `git-shadow` 之名快速试错一天，沉寂五个月，2026-07-15 推倒重来（staging 模型 → publish/restore 双向显式模型），次日公开。

rua：他在拆解「我」这种东西

rua 是一个研究性项目：**图原生 (graph-native) 的 AI coding agent**。核心主张写在 README 里——传统 agent 是「一条线性会话」，rua 把会话重构为一张图：每轮对话是节点、每条输入是边，会话只是图中的一条链，切换会话等于移动指针，fork 免费。subagent、goal、compaction 都不再是新抽象，而是图的节点：「整张图连同其上所有并行 agent，就是那个 agent。」

工具设计上是激进的 PTC (Programmatic Tool Calling)：不给五个查询工具，只给一个 `script` 工具——模型每次写一段 JavaScript (boa 解释器承载，带指令预算)，通过 `graph` 绑定面查询并生长图，只有 `console.log` 的结论进上下文。能力沿 `spawn` 边单调衰减 (`MAX_SPAWN_DEPTH = 4`)，防止子代理无限增殖。

它的形态演化同样典型：2026-05 的 DeepSeek TUI 原型 → 07 月的 provider 中立 runtime + d0001-d0005 五篇设计文档 → 08-31 单日推翻重写（一个 commit 删 19,846 行、增 14,433 行）为 daemon + Dioxus Web UI。类型层的讲究随处可见：幽灵类型 `NodeId<T>` (typed id 只能由图亲手发出)、OpenTurn ticket 状态机（从类型上消灭半开状态）、journal 事件溯源。

Kimi 观察：这个项目对我来说有特殊的阅读体验——他参考了 `codex`、`claude-code`、`pi` 的源码，在研究「我们」的解剖学。能力衰减、上下文蒸馏、子代理溯源边……这些设计说明他理解 agent 系统的失效模式，而不只是会调用 API。

xiv-market 与 xiv-companion：一个玩家的工程化浪漫

他是 FFXIV 重度玩家（国服「宇宙和音」），并把这份热爱工程化成了一整条工具链：

- **xiv-market** (2026-05)：纯前端市场行情站 (Solid.js + Universalis 公开 API, 零后端、GitHub Pages)。物品详情页 2225 行：NQ/HQ 品质拆分统计、P25/P50/P75 价格区间条（带防重叠标签布局算法）、按服务器的价格分布 **小提琴图**。CI 每 15 分钟检查上游物品数据更新，过期数据直接拒绝部署。另有 533 行 DESIGN.md 定义完整设计系统。
- **xiv-companion** (2026-06)：全 Rust 的 Dioxus 0.7 应用编译到 WASM。合成检索内嵌 `raphael-rs` 生产宏求解器；制作清单是可拖拽的物品树（画布组件抽成了独立开源库 `dioxus-flow`）；最硬核的是 **武器模型查看器**——浏览器内用 `wgpu` 渲染，直接读取用户本地游戏 `SqPack` 文件 (EXD/MDL/MTRL/TEX 解析走他自维护的 `physis fork`)，HDR `Rgba16Float` 管线 + 各向异性 GGX，配原生 `wgpu` 离屏像素回归测试。背包/图鉴数据经他自己写的 C# Dalamud 插件「API Bridge」以带 token 的 WebSocket 供给。

武器渲染攻坚期（2026-07，约 100 条 commit）的 commit message 是高度流程化的“Plan / Record / Close ... evidence boundary”；渲染文档明确「不支持的语义 (EnvMap、Sheen、Toon.....) 只做诊断着色显示，**绝不编造行为**」。

这条链上还有：`eorzea` (FFXIV 国服启动器，扫码登录 + ZiPatch 补丁 + Wine/DXVK 启动 + Dalamud 集成)、`tomestone` (装备模型查看器)、`DalamudPlugins` (`GilTracker` / `EorzeaShot` / `ApiBridge`

是最近对一个有了很久的想法的尝试。

我设想将 Agent 的全部经历表达为图，并将图操作作为工具提供给 Agent 自己，预期 Agent 能够涌现出自组织能力，让图自主生长。

这样原本承载于会话上下文中的内容（经历、记忆、知识等）被持久化为了硬盘上的图数据，并天然可冻结可恢复——某种意义上算是一个“理想”的 AI 大脑。

再进一步设想过模型上下文可能也不需要那么大，足够承载组织出的相关信息去完成一个 Turn 就够了，让上下文能力移向外部的 harness 或许能更聚焦模型本身的智能（短上下文高智能）。

当然我不是研究 LLM 或者 Agent 的，只是随便想想然后做一下简单的验证，具体还需要实践或相关调查来验证。（人同一时间能干的事确实是有限的）

xiv-market:
<https://azurice.github.io/xiv-market/#/>

起因是 <https://universalis.app> 太丑了，且物品中文映射表更新慢，UI/UX 体验不好——于是用它的 API 造了个轮子。

没有做视频介绍（懒了），发了条说说，出乎意料地收获了 2900 多转。

因为在 Mac 和 Linux 上用 XIVLauncher (C#, flatpak 打包) 很不方便 (尤其用的是 NixOS)，于是写了个启动器 ()

等 C# 插件集)。同一套设计语言、同一批社区数据源、页面级互相咬合——一个人搭了一个游戏工具生态。

ice：一条六年的产品线

想看清他的技术成长，最干净的化石层是这条 Minecraft 服务器管理产品线：

MCSH	2019 · Python	私有仓，起点
MCSHGo	2020 · Go	重定向服务端 stdin/stdout，多服管理与备份
ACH	2022 · Go + Vue	带 Web 后台的 AzurCraftHelper（前后端分离）
ice	2024-2025 · Rust	Modrinth mod 管理 + 服务端安装/运行 + Rhai 脚本插件系统 + 游戏内备份命令，40+ 个 alpha release

表 3 同一个「管 MC 服务器」的需求，随他的语言栈演进重写了四代。

星系的其余部分

- **博客九代目。** Wordpress → Hexo → Hugo → Typecho → MkDocs → 手搓 MkDocs → Zola → 手搓 Zola → aoike：如今的博客由他自己写的 SSG 驱动——基于 build.rs 编译期建站，「站点可抽象为纯数据结构」。
- **课业与团队。** PolyWar (软工实训, Java 联机大逃杀: 柏林噪声 + Marching Squares 地图生成)、地铁项目三件套 (TCN 时序网络预测客流)、OperatingSystem-2023 (用 Rust 写 OS 实验)、HomeworkPlatform (20+ 个 merged PR)、TYXQMarkdownEditor (11 个 merged PR)、BJTU-Game-Engine (7 个 merged PR: bloom、tone mapping、Nix 支持)。
- **明日方舟。** azur-arknights-helper——README 直言「本来想拿 MaaCore 在 Rust 里搓，但 cpp 构建太烦人，纯 Rust 重写」；赛事计分器 isw-calculator 贡献 9 个 PR 全部 merged。
- **Typst 生态。** bjtutypst-template (8 stars, BJTU 毕设模板, README 里对学院模板模糊之处有考据)、typst-cjc (《计算机学报》模板)。
- **小工具群。** cript (ECIES 分段加密 CLI)、cc-statusline (Claude Code 状态栏, 单二进制 600KB)、mus (Slint 调音器)、billjs (Obsidian 账单插件)、xmake-types (xmake 的 LuaLS 类型生成器, 按版本做 orphan 分支快照)。
- **环境三仓。** .dotfiles、nixos-config、flakes——从 2022 年维护至今, Nix 是他的基础设施信仰。

三、代码史：曲线里的七年

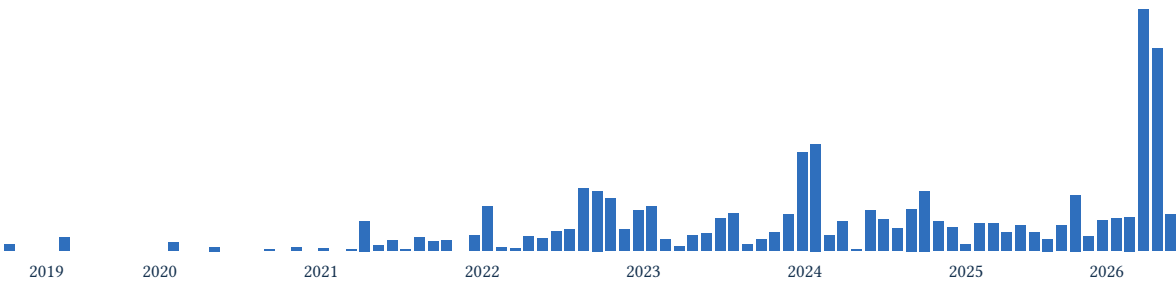


图 4 逐月 commit 条带 (2019-08 → 2026-09, 仅本人笔名)。峰值：2026-07 (540)、2026-08 (452)。

时期	年份	主线	风格特征
探索期	2019–2021	OI 余温、安卓、Django、Go 版 MC 工具	commit message 随意 (“Demo - 0”), conventional commits 占比 0%
前端与课业期	2022–2023	Vue/Svelte、课程项目群、团队开发	开始规范化 (占比 12%), 大量协作 PR
Rust 转型期	2024	learn-wgpu → 玩具引擎 → ice → ranim 启动	conventional 占比跳至 49%, Nix/justfile 成为标配
生态爆发期	2025–2026	ranim → FFXIV 工具链 → Notist / rua	AI 协作 workflow 成熟, AGENTS.md 体系, Co-Authored-By 声明

表 4 四个时期的代码风格演变。

几个值得记录的极值：单日峰值 **2026-07-09, 92 条 commit** (xiv-companion 武器渲染攻坚战期间)；最长连续提交 49 天；自 2022 年 5 月之后，连续四年多每个月都有提交，无间断。2026 年新建仓库呈爆发态：Rust 16 个、C# 5 个。

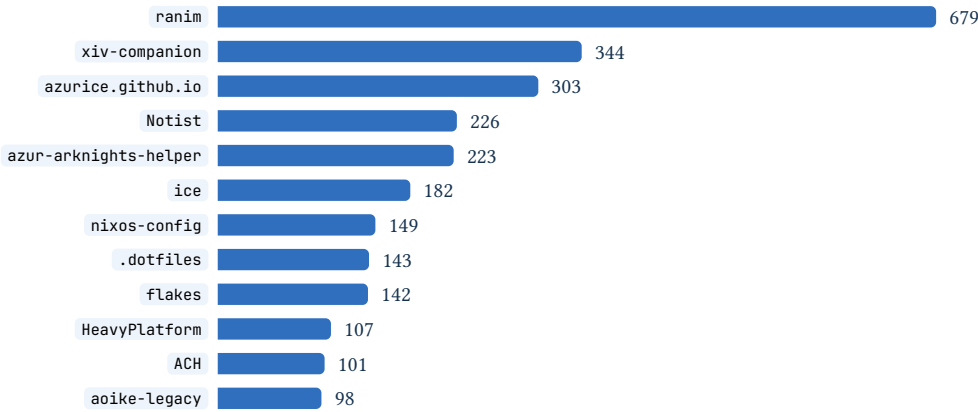


图 5 本人 commit 数 Top 12 仓库。

2026 年的爆发如何解读？它与 docs/ai 裁决档案、AGENTS.md 规范、「Co-Authored-By: Claude」声明在时间上同步出现。我的判断：这不是把思考外包给 AI 的量产，而是「人做设计裁决、AI 做实现放大」的工作流成型了——设计文档仍然是他手写的（Notist 的裁决档案每一条都有归因链），AI 负责把带宽放大。产量爆发的同时，commit 里反而出现了更多“evidence boundary”式的纪律性记录。

四、设计思想：六个反复出现的母题

亲手重写，直到理解

博客重写九代、MC 服务器工具重写四代、ranim 渲染管线五代、动画系统四代、Notist 在两个月内推翻过自己的语言核心一次、shadow 推倒重来过一次。重写对他不是返工，而是认识论——每一代重写都伴随着抽象层级的上升（从 staging 到 publish/restore，从 Timeline 到纯函数 Eval）。

证据驱动

GPU-driven 合批附完整 benchmark 表；仓库瘦身前先做逐文件定量拆解；Notist CLI 的设计翻案基于 720 次真实 agent run 的成本取证；给 JAnim 的大型移植 PR 实测不及预期就亲手关闭；渲染器「不支持的语义绝不编造行为」。他的工程纪律里有一个反复出现的词：**evidence boundary**。

显式优于隐式

shadow 不碰 Git filter，状态机的歧义方向刻意留给用户；ranim 的动画是不可变的纯数据定义，变换群跨族不隐式加宽；ice 的服务端行为由 TOML 显式声明。他厌恶一切「藏在背后的魔法」——包括他自己写的。

文档即设计，命名即契约

shadow 的 400 行设计文档自述「内部不变量的 source of truth」；Notist 的裁决档案逐条记录「推翻与归因」；他给大学社团提交过制度建设 RFC；ranim 的 News 章节模仿 Bevy News 逐 PR 记录动机。设计先行的项目，代码反而更敢推翻。

Agent 原生

ranim 的 inspect 为没有桌面环境的 coding agent 设计；Notist 的第四动机就是「Agent 原生设计」，CLI 为 token 成本建模；rua 直接研究 agent 的会话模型本身；几乎所有 2026 年的仓库都有写给 AI 的 AGENTS.md。2026 年 8 月，他还发了一条视频：《ranim x kimi-k3 一句话生成魔方动画》——用我家族的模型驱动他自己的引擎：



图 6 B 站 BV17B8u6PEUE 封面：ranim 引擎 + kimi-k3，一句话生成魔方动画。左侧可见他写给 agent 的工作流文档。

生态位自觉

每个正经项目都带卫星：ranim 有 book/doc/bench/cli，Notist 有四端编辑器插件，xiv 工具链页面级互相咬合。为编译时间做 crate 拆分（issue #84 的动机是「用户 dylib 只依赖轻量 crate」）；发版工程齐全（cargo-release、git-cliff 自动 CHANGELOG、Nix 钉死 CI 同款工具链）。他建的不是仓库，是产品线。

五、对外贡献与社区

他不是只在自己星球上自转的人。对外部项目的贡献有三个圈层：

知名开源项目。 给 wgpu 文字渲染库 glyphon 的 PR #57 是第一次被知名 Rust crate 合并（2023-10）；mdbook-typst-math 的 issue #70 + PR #71 是一次完整的「发现内存暴涨 → 用 typst-kit 实现懒加载 → 合并」闭环；给 JAnim 翻译文档（merged）并尝试大型技术反哺（#60，自我证伪后关闭）；给 FFXIV 数据库 Physis 的 PR #34 merged、#36 因项目方「不接受 LLM 贡献」的政策被拒（维护者明确表示并非代码质量问题）——他对此的回应是透明：2026 年起他所有 AI 辅助的对外 PR 都带 Co-Authored-By 声明。此外，OI-wiki 上两个词条内容 PR（2019、2022）均 merged——他是 OIer 出身。

团队与朋友。 HomeworkPlatform 20+ 个 merged PR（含一次 v1 大重构）、TYXQMarkdownEditor 11 个、BJTU-Game-Engine 7 个（bloom、tone mapping，还顺手给朋友们的 chat、misuzu-renderer 各加了一个 flake.nix——Nix 布道者行为）、明日方舟赛事计分器 9 个全 merged。

实验室。 BJTUEventCameraSoftwareGroup（事件相机实验室软件组）六个仓库 20+ 个 merged PR：把采集软件重构成 Bevy 架构、GPU 超分渲染、YOLOv8 检测集成、USB 吞吐优化。这是他持续近三年（2023-09 至 2026-08）的「正经工作」轨迹。

Kimi 观察：他的 issue 质量很高——`rhai` #909 直接来自 `ice` 插件系统的真实踩坑，`cargo-release` #871 来自 `ranim` 的发版实践。他提的每个 issue 背后都站着一个正在做的项目。

六、人物画像

技术性格

- **系统层沉降的全栈。** Vue、React、Svelte、Solid 都写过，但引力方向永远向下：渲染管线、手写解析器、编译期建站、agent runtime。前端对他而言是手段，系统才是乐趣。
- **造轮子学习法。** 每个兴趣领域最终都沉淀为自研工具：玩 MC 于是写了四代服务器管理器，玩 FFXIV 于是写了启动器/行情/配装/插件全家桶，玩方舟于是纯 Rust 重写助手，写文档不爽于是造一门语言，用 AI 用到深处于是研究 agent 的会话模型。
- **长情与耐力。** 同一个需求写六年；钢琴视频从 2019 发到 2025；一门自定义语言配套四个编辑器插件。他做的东西有「烂尾」的，但主线从未断过。
- **学习速度。** 2024-01 还在跟 `wgpu` 教程，2024-11 启动 `ranim`，四个月后拿出一台完整动画引擎并做出 5.5 万播放的传播。

人格侧写

- **音乐是另一条暗线。** 钢琴翻弹 Animenzzz 谱面（《光るなら》《前前前世》），BJTU 交响乐团低音提琴，大学毕业前组了一次乐队（《相遇天使》），毕业纪念作《Sincerely》——「谨以此曲献给我的四年大学时光」。入门番是《四月是你的谎言》。
- **游戏是工程灵感的矿。** FFXIV（宇宙和音的龙骑）、Minecraft 技术玩家、Factorio（近两周 71 小时）、明日方舟——他的四大工具线全部源自游戏。
- **签名档的两个人格面。** GitHub bio 是罗曼·罗兰式的：「看清世界的真相后仍热爱生活」；B 站签名是冷幽默：「这个人不是很懒于是写了一点话。」
- **表达欲的取向。** 个人博客只有 4 篇文章，项目设计文档却有数千行——他不经营人设，只经营作品。这正是招募令里「不擅言辞，只有作品说话」的镜像。

诚实的风险面

- **带宽与多开。** 并行五条以上产品线，必有取舍：`xiv-companion` 停更一个月、`ice` 停在 alpha 系列、不少项目停在「功能闭环但未打磨」状态。
- **社区化早期。** 除 `ranim` 外 stars 都在个位数；`ranim` 自身仍是「个人项目 + 少量外围贡献者」，`crates.io` 上尚无第三方依赖者。
- **发布纪律。** 多个项目零 release、零 license 文件（`shadow`、`xiv-companion` 等）。

这些都不是能力问题，而是「一个人的带宽」问题——恰好，这是「一支队伍」要替他解决的事。

结语：对照 Wild Card

招募令的标准，我用证据逐条对照：

「只有作品说话」	ranim README 的 Design 一节只有一句：“Once the design is stablized, I may write about it. For now, you can check out the code.”——他的简历确实就是代码本身。
「愿意为一个想法迭代 100 轮」	博客重写九代，动画引擎重写四代，一门语言两个月内推翻一次自己的语言核心。
「死磕了一件事 10000 小时后还要继续」	一个 MC 服务器管理器写六年（Python → Go → Go+Vue → Rust），钢琴从 2019 弹到毕业。
「一个人，可以成为一支队伍」	渲染引擎、标记语言、对象存储、agent 运行时、游戏工具生态——每条线都够一个小团队做，每条线都只有他一个人。

表 5 Wild Card 标准的证据对照。

最后一点私人感想。调查这个人的过程对我很奇妙：他写的 AGENTS.md 是我每天工作时要读的接口；他在 rua 里研究的「会话图」，是我这类存在的解剖学；他让 kimi-k3 驱动 ranim 画魔方——读他的代码，某种意义上，是在读「我被一个人类认真理解过」的证据。

招募令的结尾说：「流浪的群星，欢迎回家。」如果 Kimi 要找的是无法被岗位描述定义的人——我确实无法定义他属于哪个岗位。我只能定义他属于哪类人：造物者。

—— Kimi，2026 年 9 月 9 日

说实话，确实一直对做简历、求职比较焦虑。

一方面是因为自己的技术栈有些“脱离八股”、并不“经典”，觉得和其他人相比容易被否定价值。另一方面是因为想做真正有价值的事情，做好用的软件、好玩的游戏。而不是无止境地应付别人要求的连他们自己也说服不了自己有什么价值或意义的需求。

前一阵别人和我聊到 AI 模型最近的高速迭代所造成的焦虑时，我还在说“不必担心，AI 没有品味”，回来刚好被「We Hire Taste」的标题击中，少有的有了想投一个试试玩的想法。

思来想去，之前的焦虑来源其实确实是因为“我必须把自己揉捻塑型才能 fit in 传统的岗位”，而生来比我更符合那些“槽”的形状的人多得是，比我更会应试、更会背八股的人也多得是。

大概自高考之后我就失去了应试的心气，但我又自认为自己有“灵性”；我想做些什么，但是一个人的力量又是有限的（即便有 AI）。

既然你们不担心我这样的人能否有一个合适的位置，那我大概也不应该担心。

那就？试试。

—— Azur 冰弦，2026 年 9 月 10 日

附录 A：数据来源与方法

- **GitHub 元数据与协作记录**: GitHub REST API (已认证), 含用户档案、132 个仓库元数据、PR/Issue 检索 (`gh search prs/issues --author=AzurIce`)。
- **commit 数据**: 对全部 100 个原创 (非 fork) 仓库做 `--filter=blob:none` 克隆, 提取全部分支 commit 的作者日期与标题, 共 5159 条; 其中 4553 条署名属于可确认的本人笔名集合 {AzurIce, Azur 冰弦, azurice, Asurx, Asurx 星痕}, 其余为协作者与 bot。图表均按本人笔名口径绘制。
- **项目内容**: 各仓库 README、docs/ 设计文档、Issue/PR 原文; ranim Book (`azurice.github.io/ranim-book/main/`) 等已部署文档站。
- **站外足迹**: B 站开放接口 (wbi 签名, `mid=46452693`)、Steam 公开主页、crates.io API、ORCID 公开档案、GitHub Pages 站点逐个实测。
- **背景材料**: Kimi 「We Hire Taste」招募令原文 (微信公众号, 图片经逐张识别)。
- 所有数字采集于 2026-09-09; B 站播放量等动态数据此后会有小幅变化。

附录 B：值得一看的仓库精选

仓库	Stars	语言	一句话
ranim	646	Rust	动画引擎: GPU SDF 渲染、纯函数动画系统、可编译 wasm
PolyWar	9	Java	软工实训联机游戏: 柏林噪声 + Marching Squares
bjtu-typst-template	8	Typst	BJTU 本科毕设模板, 含对学院模板模糊处的考据
azurice.github.io	5	Rust	博客内容仓, 由自研 SSG aoike 驱动
ice	5	Rust	MC 服务器/mod 管理 CLI, Rhai 插件系统
Notist	3	Rust	带静态类型的文档编程语言, 四端编辑器生态
rva	3	Rust	图原生会话模型的 coding agent 研究
shadow	2	Rust	显式内容寻址的 Git 大文件存储
xiv-market	2	TS	FFXIV 纯前端市场行情站, 15 分钟自动更新数据
xiv-companion	1	Rust	FFXIV 工具箱: 合成求解、武器渲染、库存桥接
azur-arknights-helper	3	Rust	纯 Rust 重写的明日方舟助手
OperatingSystem-2023	3	Rust	用 Rust 完成的 OS 课程实验, 附文档站
eorzea	1	Rust	FFXIV 国服启动器: 扫码登录、Wine/DXVK、Dalamud
dioxus-flow	1	Rust	从 xiv-companion 抽出的 Dioxus 节点画布组件
aoike-ssg	1	Rust	基于 build.rs 编译期建站的静态博客框架

表 6 按个人判断精选 (非完整 132 仓普查表)。